

.

```
def kolonin_motivation_pipeline(  
    user_message: str,  
    llm_client,  
    current_state: Optional[MotivationState] =  
    None  
) -> Dict:  
    """Полный пайплайн обработки с учетом  
    мотивации Колонина"""  
  
    # Инициализация состояния  
    if current_state is None:  
        state =  
        MotivationState(needs=init_kolonin_needs())  
    else:  
        state = current_state  
  
    # 1. Анализ мотивации  
    state = update_motivation_state(state,  
    user_message)
```

2. Стратегия ответа

```
strategy = get_response_strategy(state)
```

3. Формирование промпта для LLM

```
system_prompt = f"""
```

Ты используешь модель мотивации
Колонина для адаптации ответов.

ТЕКУЩЕЕ МОТИВАЦИОННОЕ СОСТОЯНИЕ
ПОЛЬЗОВАТЕЛЯ:

- Доминирующая потребность:

```
{state.dominant_need or 'не определена'}
```

- Класс активности:

```
{state.activity_focus.value if  
state.activity_focus else 'не определен'}
```

- Срочность: {state.urgency:.2f}

СТРАТЕГИЯ ОТВЕТА:

```
{strategy['prompt']}
```

ПРАВИЛА:

- Фокусируйся на {strategy['focus']}

- Используй стиль: {strategy['style']}
- Поддерживай
{state.dominant_need.replace('_', ' ').title()}
пользователя
- Избегай фрустрации других потребностей
""

```
messages = [  
    {"role": "system", "content":  
system_prompt},  
    {"role": "user", "content": user_message}  
]
```

4. Генерация ответа

```
response =  
llm_client.chat.completions.create(  
    model="gpt-4",  
    messages=messages,  
    temperature=0.6  
)
```

```
result = {
```

```
    "motivation_state": state,  
    "strategy": strategy,  
    "response":  
response.choices[0].message.content,  
    "dominant_need": state.dominant_need,  
    "activity_focus": state.activity_focus  
}  
  
return result
```